

LIMO Warehouse System - Developer Documentation

1. Project Overview

****Project Name****

- LIMO Warehouse System

****Purpose of the system****

- A browser-based warehouse inventory management prototype for inbound and outbound cargo operations.
- Supports shipment tracking, inventory monitoring, user authentication, and QR code scanning.

****Objectives****

- Provide a simple warehouse operations dashboard for employees and administrators.
- Enable inbound and outbound shipment registration.
- Present inventory and activity reports.
- Support user management, profile management, and notifications.

****Main features****

- Secure login screen with password visibility toggle.
- Role-based page access and admin-only user management.
- Dashboard with shipment statistics.
- Inventory list with filtering, sorting, pagination, and shipment details drawer.
- Inbound/outbound forms with shipment capture and auto-complete assistance.
- QR/Barcode scanning support via camera preview and jsQR.
- Profile editing with name/email sync and profile picture upload.
- Local Storage persistence for application data.

****Intended users****

- Admin: manages users, reviews inventory, sees all reports.
- Employee: performs inbound/outbound operations, views inventory, profile, and notifications.

2. Technology Stack

| Technology | Role | Why Used |

|---|---|---|

| HTML5 | Structure | Provides semantic layout for pages and modals.

| CSS3 | Styling | Controls responsive UI, layout, and animations.

| JavaScript (Vanilla) | Application logic | Handles state, page rendering, auth, UI interactions, and data management without a framework.

| Bootstrap Icons | Iconography | Provides icons for buttons, notifications, navigation, and UI affordances.

| Local Storage | Persistence | Temporary data store for auth accounts, session, shipments, profile, and notifications.

| jsQR | QR Scanner | Reads QR code payloads from camera frames.

| Camera API (`navigator.mediaDevices.getUserMedia`) | Camera access | Captures live video for QR scanning.

| xlsx | Excel export | Provides front-end export capability for data.

| jsPDF + jsPDF-AutoTable | PDF export | Generates printable PDF output from data tables.

****Note**:** No backend, server-side code, or real database is included. The system is fully front-end and uses browser storage.

3. Folder Structure

/ (root)

index.html

README.md

styles.css

assets/

app.css

app.js

auth.js

login.js

login_bg.png

logo-YL.png

logo.svg

pages/

dashboard.html

inbound-outbound.html

inventory.html

profile.html

user-management.html

tests/

auth.test.js

****File and folder purpose****

- ``index.html``: Login screen and entry point.
- ``README.md``: Developer documentation and project handover.
- ``styles.css``: Login page styles and responsive layout for the landing page.
- ``assets/app.css``: Shared application styles for all authenticated pages.
- ``assets/app.js``: Main app logic, page initialization, UI behavior, storage, and feature implementation.
- ``assets/auth.js``: Authentication service, account management, session handling, and admin operations.
- ``assets/login.js``: Login page form handling, validation, and redirect.
- ``assets/login_bg.png``, ``assets/logo-YL.png``, ``assets/logo.svg``: Visual assets.
- ``pages/*.html``: Authenticated application pages for dashboard, inventory, inbound/outbound shipment capture, profile management, and user management.
- ``tests/auth.test.js``: Unit-style tests for ``AuthService`` in ``assets/auth.js``.

4. System Architecture

Login flow

1. User opens ``index.html``.
2. ``assets/login.js`` checks ``AuthService.isAuthenticated()``.
3. If already logged in, the user is redirected to ``pages/dashboard.html``.
4. On form submission, ``AuthService.authenticateUser()`` validates credentials.
5. Successful login persists session to Local Storage and redirects to dashboard.
6. Failed login displays an error in the UI.

User authentication

- ``assets/auth.js`` stores accounts in Local Storage under the key ``warehouseAuthAccounts``.
- Admin and employee accounts are seeded automatically if no auth data exists.
- Session data is stored under ``warehouseAuthSession``.
- ``AuthService`` exposes ``isAuthenticated()``, ``getCurrentUser()``, ``authenticateUser()``, and ``logout()``.

Admin role

- Admins can access ``pages/user-management.html``.
- ``assets/app.js`` checks the URL against ``isAdminRoute`` and denies access for non-admins.
- Admin-only navigation links use ``admin-only-nav``.

Employee role

- Employees can access all regular pages except admin-only user management.
- The role is determined from current session.

Navigation flow

- Sidebar navigation is present on authenticated pages.
- The top bar includes search, notifications, and profile dropdown.
- User dropdown contains links to Profile, User Management (admin only), and Logout.
- Page transitions are handled by standard anchor links pointing to HTML pages.

Data flow

- Shipment data is loaded from Local Storage using ``getStoredShipments()`.`
- Data is normalized and aggregated with helper functions (``aggregateShipmentsByReference`,` ``normalizeShipmentForInventory`,` ``normalizeShipmentForActivity``).
- Inventory and activity pages consume aggregated shipment records.
- Profile data is loaded and updated from Local Storage separately.
- Notifications are generated from recent shipment records and persisted in Local Storage.

5. Features Implemented

Dashboard

- Displays totals:
 - ``Total Shipments``
 - ``Cargo in Warehouse``
 - ``Outgoing Today``
- Clickable cards filter the activity report view.
- Recent shipments section appears if present.

Inventory

- Search by HAWB, MAWB, or client.
- Filters:
 - Warehouse Location
 - Transaction Type
- Sorting:
 - Remaining Quantity ascending/descending.
- Page size controls (5, 10, 20, All).
- QR scan button opens the camera scanner modal.
- Shipment details drawer shows detailed shipment metadata.

Inbound & Outbound

- Tabbed forms for inbound and outbound shipments.

- Manual shipment fields include client, destination, HAWB, MAWB, invoice, transaction type, and location.
- Receiving and release information capture dates, times, drivers, trucks, plate numbers, and cargo condition.
- Quantity section supports multiple quantity rows and unit selection.
- The form saves shipment entries to Local Storage.

Profile

- Displays current user profile information.
- Inline editing for full name, username, and email.
- Profile picture upload and basic cropping support.
- Uses Local Storage for profile and picture persistence.

User Management

- Admin-only user table.
- Search by employee ID, name, or username.
- Role filter: admin / employee.
- Add user modal.
- Edit and view user records.
- Reset password to default (`Password123`).
- Toggle user status active/inactive.
- Delete users (protected from deleting current admin or last admin).

Notifications

- Notification bell with live state.
- Generated from recent shipment activity.
- Unread/read state persisted in Local Storage.
- Supports marking all as read and individual notification read actions.

6. QR Code Module

Camera API

- The app uses `navigator.mediaDevices.getUserMedia` to access the device camera.
- Live video is rendered in a `` element inside `scanModal`.

jsQR scanning process

- `jsQR` is loaded from CDN in authenticated pages.
- The camera frame is captured to a canvas and scanned using `jsQR`.
- If a code is detected, the decoded payload is parsed.

JSON payload format

- The system expects a JSON string inside the QR code payload.
- The payload may contain shipment fields such as HAWB, MAWB, client, transactionType, location, and other relevant data.

Parsing

- The scanner parses JSON from code data.
- If valid, the values are mapped into form fields.
- If invalid payload occurs, the scanner logs warnings and may prompt users to retry.

Auto-filling form fields

- When a HAWB/MAWB QR is recognized, the value is auto-populated in the form.
- Outbound HAWB autocomplete also helps auto-fill inbound fields from existing inventory.

Recommended future improvement

- Current implementation may store full shipment data in the QR code payload.
- Future enhancement: QR codes should store only the HAWB identifier, and the system should fetch complete shipment details from the database by HAWB.

7. Data Storage

Local Storage structure

- `warehouseAuthAccounts`: array of user accounts.
- `warehouseAuthSession`: current session object.
- `warehouseShipments`: array of shipment records.
- `warehouseProfile`: profile metadata.
- `warehouseProfilePictures`: object keyed by userId with data URL values.
- `warehouseNotificationState`: array of notification objects.
- `sidebarCollapsed`: layout preference.
- `warehouseLiveSearchSelection`: temporary search selection state.

Keys used

- `warehouseAuthAccounts`
- `warehouseAuthSession`
- `warehouseShipments`
- `warehouseProfile`
- `warehouseProfilePictures`
- `warehouseNotificationState`

- `sidebarCollapsed`
- `warehouseLiveSearchSelection`

Objects and arrays

- User account objects include: `userId`, `password`, `role`, `name`, `employeeId`, `position`, `status`, `lastLogin`, `createdAt`.
- Shipment records include `client`, `destination`, `hawb`, `mawb`, `invoice`, `transactionType`, `date`, `time`, `quantity`, `unit`, `location`, `status`, `entryType`, `releaseDate`, `releaseTime`, `releasePlate`, `releaseDriver`, `remarks`, and `savedAt`.
- Profile objects include `firstName`, `lastName`, `employeeId`, `position`, `email`, `phone`, plus saved username.
- Notification state array stores items with `id`, `type`, `title`, `description`, `meta`, `time`, and `read`.

Relationships

- Shipments are grouped by HAWB or MAWB to aggregate inventory and activity records.
- Users are independent of shipments; session state links current user to actions.
- Profile data is keyed by userId, allowing personalization per logged-in account.

Migration to database

- Replace Local Storage logic with API calls to a backend database.
- Use tables for `users`, `sessions`, `shipments`, `profiles`, `notifications`
- Use foreign keys:
 - `profiles.user\_id` references `users.id`
 - `shipments.created\_by` references `users.id`
 - For Supabase/PostgreSQL:
 - use `auth.users` or manual user table
 - store session in JWT or server session cookie
 - move `warehouseShipments` to normalized `shipments` table with inbound/outbound rows.
 - Add server-side validation, authentication, and data migration scripts.

8. Business Logic

Inventory calculation

- Inventory is aggregated by HAWB/MAWB reference.
- `remainingValue` = `qtyInValue` - `qtyOutValue`.
- `qtyInValue` and `qtyOutValue` are parsed from numeric quantity fields.
- Remaining inventory uses units from the latest shipment record.

Remaining Quantity

- Shown in inventory as formatted quantity string.
- If outbound exceeds inbound, remaining quantity may be zero or negative depending on raw data.

Partial outbound

- Outbound entries can be recorded separately from inbound.
- `partialFull` and `status` fields distinguish full or partial release.
- The aggregated record preserves multiple outbound details.

Multiple outbound transactions for one HAWB

- `aggregateShipmentsByReference` groups entries by HAWB or MAWB.
- Outbound details are stored in `outboundDetails` array.
- Latest release date/time/plate/driver is preserved as final release info.

Status updates

- Status is drawn from shipment records, or default `WAITING FOR CONFIRMATION`.
- Badge color logic is based on status strings containing `RELEASE` or `RECEIVED`.

Dashboard statistics

- `Total Shipments`: count of aggregated shipment records.
- `Cargo in Warehouse`: sum of remaining quantities across grouped shipments.
- `Outgoing Today`: counts shipments with outbound details or release date equal to today.

Activity history

- Activity records are normalized separately for display.
- Each activity record includes month, client, HAWB, MAWB, date in/out, quantity in/out, and badge type.
- Filters include search, date range, location, and transaction type.

Search algorithm

- Inventory and activity search uses substring matching on HAWB/MAWB/client.
- Global topbar search also searches inventory and activity items.
- User management search filters employee ID, name, or username.

Filters

- Inventory filters: location, transaction type.
- Activity filters: search term, date range, transaction type, location.
- Dashboard cards set the active activity filter for a quick view.

Sorting

- Inventory sort by remaining quantity ascending/descending.

- User management sort by employee ID, name, username, role, status, last login.

9. UI Components

Cards

- Dashboard uses cards for totals and metrics.
- Clickable cards highlight active filter state.

Tables

- Inventory and activity pages use HTML tables for row display.
- User management uses a user table with action buttons.

Filter modal / panel

- Inventory and activity pages each have a collapsible filter panel.
- Date range uses a popover calendar and preset buttons.

QR Scan modal

- `scanModal` appears on inventory and inbound/outbound pages.
- Includes camera preview, scanning overlay, cancel button, and instructions.

Notification popup

- Custom notification panel appears when clicking bell icon.
- Includes unread badge and interactive read actions.

Side drawer

- Inventory page has a slide-out drawer for shipment details.
- Drawer content populates multiple record sections.

Profile modal

- Profile page includes inline editing.
- Profile picture modal supports upload, zoom, drag, rotate, and save.

Buttons

- Buttons are styled from `app.css`.
- Primary, secondary, icon, and action buttons have distinct styles.

Responsive mobile layout

- Auth page uses responsive grid in `styles.css`.
- `app.css` likely includes responsive rules for authenticated pages.

10. Responsive Design

Desktop layout

- Sidebar navigation on the left.
- Top bar with search and profile.
- Wide content area for tables and forms.

Tablet layout

- Sidebar collapses or becomes icon-only.
- Content adapts to narrower width.

Mobile layout

- Sidebar collapses into a toggle.
- Filters and search stack vertically.
- Login page becomes single-column.

Breakpoints

- `styles.css` shows breakpoints at `920px` and `680px` for login layout.
- `app.css` likely includes additional breakpoints for authenticated pages.

Navigation changes

- Sidebar toggle persists collapsed state in Local Storage.
- User dropdown and notifications use overlays to remain accessible.

11. Future Improvements

- Replace Local Storage with a real database and backend API.
- Implement role-based permissions beyond admin/employee.
- Add barcode scanning support in addition to QR codes.
- Audit logs for each inbound/outbound action.
- Excel import/CSV upload for bulk shipment entry.
- Improve PDF export styling and support custom report formats.
- Real-time dashboard updates using WebSocket or polling.
- Integrate with cloud deployment (Netlify, Vercel, AWS, Supabase).

- Automatic backup and export of Local Storage data.
- Add server-side validation and secure password hashing.
- Add full unit/integration testing for UI and backend.
- Convert to a reactive framework for maintainability (React/Vue/Svelte).

12. Setup Guide

Required software

- Modern web browser (Chrome, Edge, Firefox, Safari).
- Local development environment with file server support if needed.
- Node.js only required if running test script locally.

How to run the project

1. Open the workspace folder in VS Code.
2. Open `index.html` in a browser, or use a local file server.
3. No build step is required, as the project is pure static front-end.

Browser requirements

- Supports `localStorage` and `sessionStorage`.
- Supports `navigator.mediaDevices.getUserMedia` for QR scanning.
- Supports ES6 JavaScript and modern DOM APIs.

Folder setup

- Keep the `assets`, `pages`, and root `.html` files together.
- Do not rename `assets/auth.js` or `assets/app.js` without updating references in HTML.

Dependencies

- External libraries are loaded through CDN in HTML pages: `jsQR`, `xlsx`, `jsPDF`, `jsPDF-autotable`.
- No local NPM install required.

How to test QR scanning

1. Open a page with the scan modal (`inventory.html` or `inbound-outbound.html`).
2. Click the QR scan button.
3. Grant camera permission.
4. Present a QR code to the camera.
5. Verify decoded values auto-fill fields.

13. Code Comments

Comments added in code

- ``assets/login.js`` already has clear logic and short helper functions.
- ``assets/auth.js`` includes data normalization and role guard logic, and should remain unchanged logically.
- ``assets/app.js`` implements page behavior, storage migration, aggregation, and UI interactions.

****Note**:** To improve readability, comments should be added around the following areas:

- ``guardPageAccess`` and role route guard.
- Plate/trucker autocomplete logic.
- Local Storage get/save helper functions.
- Aggregation and inventory normalization logic.
- Profile picture cropper functions.
- Notification panel and user management modals.

14. Maintenance Notes

Files responsible for each feature

- ``index.html`` + ``assets/login.js``: Login screen and sign-in flow.
- ``assets/auth.js``: Authentication, user management, password reset, session persistence.
- ``assets/app.js``: Main application features, page initialization, data storage, notifications, autocomplete, and page-specific logic.
- ``pages/dashboard.html``: Dashboard UI and activity toolbar.
- ``pages/inventory.html``: Inventory table, filters, details drawer, QR scan modal.
- ``pages/inbound-outbound.html``: Inbound/outbound shipment capture forms.
- ``pages/profile.html``: Profile display and picture management.
- ``pages/user-management.html``: Admin user list and actions.

Functions responsible for major operations

- ``AuthService.initializeAuth()``: seed and read accounts.
- ``AuthService.authenticateUser()``: validates login and persists session.
- ``getStoredShipments()``: reads shipments from Local Storage and migrates legacy entries.
- ``saveStoredShipments()``: writes shipment data back.
- ``aggregateShipmentsByReference()``: groups shipment records.
- ``normalizeShipmentForInventory()`` and ``normalizeShipmentForActivity()``: build display models.
- ``renderDashboardData()``: updates dashboard counters.
- ``attachPlateAutocomplete()`` / ``attachTextAutocomplete()``: input suggestions.

- ``applyProfilePictureToAvatar()``: update avatar appearance.
- ``openUserModal()``: add/edit/view user modal UI.

Global variables

- ``STORAGE_KEY``: shipment storage key.
- ``refreshInventory``, ``refreshActivity``: refresh callbacks.
- ``inventoryData``, ``activityData``, ``inMemoryShipmentStore``: in-memory caches.
- ``PLATE_TRUCKER_REFERENCE``, ``PLATE_TRUCKER_INDEX``: plate-to-trucker reference.
- ``TEXT_FIELD_SUGGESTIONS``: autocomplete suggestions.

Event listeners

- Login form submission in ``assets/login.js``.
- ``DOMContentLoaded`` bootstraps page behavior in ``assets/app.js``.
- Notification toggle, sidebar toggle, user dropdown controls.
- Inventory/activity filter toggle and clear filters.
- Global search suggestions on topbar input.
- Modal open/close and profile picture cropper events.
- User management actions and table sorting.

Local Storage keys

- ``warehouseAuthAccounts``
- ``warehouseAuthSession``
- ``warehouseShipments``
- ``warehouseProfile``
- ``warehouseProfilePictures``
- ``warehouseNotificationState``
- ``sidebarCollapsed``
- ``warehouseLiveSearchSelection``

Components with dependencies

- ``assets/app.js`` depends on ``AuthService`` from ``assets/auth.js``.
- HTML pages load ``auth.js`` before ``app.js``.
- ``scanModal`` markup is used by ``assets/app.js`` for QR scanning.
- ``inventory.html`` drawer content is populated by ``app.js`` and should not be modified without updating selector IDs.
- User management modal creation depends on table structure and ``AuthService`` APIs.

15. Known Limitations

Current Implementation

- The system is a static front-end app with no backend.
- Local Storage is used instead of a database.
- Authentication is demo-style and stores passwords in plaintext.
- There is no server-side validation.
- QR scanning depends on camera access and may fail on unsupported browsers.
- QR codes may store full shipment information rather than an identifier.
- Some UI pages are placeholders or have limited functionality (e.g. `shipment-details.html` has static sample content).
- Data export is included but may not be fully validated or styled.

Recommended Enhancement

- Migrate to secure backend with hashed passwords, user authentication, and server-side authorization.
- Use a database for shipments, user profiles, and notifications.
- Add stronger validation and error handling for inbound/outbound forms.
- Improve QR/scan module to support multiple code formats and fallback logic.
- Add more tests for UI flows and data processing.

16. Additional Notes

- Seed accounts:
 - `admin001` / `Admin@123`
 - `emp001` / `Employee@123`
- Default password after reset: `Password123`.
- The dashboard and inventory pages currently use generated seed data when real shipment data is absent.
- The system uses a mix of inline HTML and JavaScript-generated modals, so changes to IDs and markup should be coordinated.

Handover summary

This document captures the system architecture, pages, storage, and feature responsibilities for the LIMO Warehouse System. The next developer should begin by reviewing `assets/auth.js` and `assets/app.js`, as these files contain the core authentication and application logic. For database migration, move the Local Storage helpers to API wrappers and replace `warehouseShipments` persistence with server-backed storage.